



okay so my stack would be react js for frontend and mernstack for backend with microservices and usign load balancing using nginx and all the cool system desic systems and for the ai part i want to keet it free as possible duign development can u help me with the folder structure adn the proecess of developemnt in depth

Here's a complete **folder structure** and **step-by-step development process** for your AI Rap Battle application with microservices architecture:

Folder Structure

```
rap-battle-platform/
├── frontend/                                # React.js Frontend
│   ├── public/
│   └── src/
│       ├── components/
│       │   ├── Battle/
│       │   │   ├── BattleArena.jsx
│       │   │   ├── RapInput.jsx
│       │   │   ├── AIResponse.jsx
│       │   │   └── ScoreDisplay.jsx
│       │   ├── Auth/
│       │   │   ├── Login.jsx
│       │   │   └── Register.jsx
│       │   ├── Leaderboard/
│       │   │   └── LeaderboardTable.jsx
│       │   └── MusicGenerator/
│       │       └── BeatSelector.jsx
│       ├── pages/
│       │   ├── Home.jsx
│       │   ├── Battle.jsx
│       │   ├── Profile.jsx
│       │   └── Tournaments.jsx
│       ├── services/
│       │   ├── api.js                    # API calls to backend
│       │   └── socket.js                # WebSocket connection
│       ├── App.jsx
│       └── index.js
├── package.json
└── Dockerfile
```

[illegible]

```

├── controllers/
│   └── musicController.js
├── services/
│   └── musicGenerator.js
├── server.js
├── package.json
├── Dockerfile
├── leaderboard-service/           # Leaderboard & Stats
│   ├── src/
│   │   ├── controllers/
│   │   │   └── leaderboardController.js
│   │   ├── models/
│   │   │   └── Leaderboard.js
│   │   ├── routes/
│   │   │   └── leaderboardRoutes.js
│   │   ├── services/
│   │   │   ├── redis.js           # Redis caching
│   │   │   └── kafka.js          # Kafka consumer
│   │   └── server.js
│   ├── package.json
│   └── Dockerfile
├── infrastructure/               # DevOps Configuration
│   ├── docker-compose.yml        # Local development
│   ├── kubernetes/
│   │   ├── deployments/
│   │   │   ├── auth-deployment.yaml
│   │   │   ├── battle-deployment.yaml
│   │   │   ├── ai-lyric-deployment.yaml
│   │   │   └── nginx-deployment.yaml
│   │   ├── services/
│   │   │   ├── auth-service.yaml
│   │   │   └── battle-service.yaml
│   │   └── configmaps/
│   │       └── nginx-config.yaml
│   ├── nginx/
│   │   └── nginx.conf
├── shared/                       # Shared utilities
│   ├── utils/
│   └── constants/
├── .gitignore
├── docker-compose.yml
└── README.md

```

Free AI Tools for Development

Text Generation (Rap Lyrics): Use **HuggingChat** with LLaMA 4 or Mistral models—completely free and open source. You can also use **Groq API** (free tier gives 14,400 requests/day with fast inference). For local development, run **Ollama** with LLaMA 3.2 on your machine.^{[1] [2] [3]}

Music Generation: Use **Mubert API** (free tier available) or **Suno AI** (limited free generations). For fully free options, use **Riffusion** (open source) or **MusicGen by Meta** (can run locally).^[4]

Voice Synthesis: Use **Coqui TTS** (open source, runs locally) or **Bark** by Suno (free and open source). ElevenLabs has a free tier with 10,000 characters/month. ^[5]

Scoring/Analysis: Use **Groq API with LLaMA 3** or **HuggingChat** to analyze rap quality, rhyme schemes, and flow. ^{[2] [1]}

Step-by-Step Development Process

Phase 1: Setup Foundation (Week 1)

Day 1-2: Initialize Project Structure

Create the folder structure above and initialize Git repository. Set up separate `package.json` for each microservice and frontend. Create a root `docker-compose.yml` for local development. ^{[6] [7] [8]}

Day 3-4: Setup MongoDB, Redis, RabbitMQ, Kafka

Use Docker Compose to run MongoDB, Redis, RabbitMQ, and Kafka locally. Create database connections in shared utilities. Test connections from each service. ^{[7] [9]}

Day 5-7: Build Auth Service

Create JWT-based authentication with register, login, and token refresh. Connect to MongoDB for user storage. Add password hashing with `bcrypt`. Test with Postman/Thunder Client. ^[8]

Phase 2: Core Battle System (Week 2-3)

Week 2: Battle Service + RabbitMQ

Create battle creation, turn management, and battle history endpoints. Integrate **RabbitMQ** for real-time turn-based messaging—when user submits rap, publish to queue. Set up battle state management in MongoDB. Add WebSocket using Socket.IO for real-time updates to frontend. ^[7]

Week 3: AI Lyric Service (Python)

Build Flask/FastAPI service that consumes RabbitMQ messages. Integrate **Groq API with LLaMA 3** for free rap generation (14,400 requests/day free). Create rhyme analysis using **Pronouncing** library (open source). Publish AI responses back to RabbitMQ for battle service to consume. ^{[3] [7]}

Phase 3: Kafka Event Streaming (Week 4)

Kafka Setup

Install Kafka producer in battle service to stream all battle events (user rap, AI response, scores). Create Kafka topics: `battle-events`, `scoring-events`, `leaderboard-updates`. ^[7]

Scoring Service

Build service that consumes battle events from Kafka. Use **Groq API** to analyze rap quality (flow, rhymes, punchlines) and assign scores 1-10. Publish scores back to Kafka for leaderboard updates. ^{[9] [3]}

Leaderboard Service

Consume scoring events from Kafka. Update user rankings in MongoDB and cache top 100 in

Redis for fast retrieval. Expose endpoints for global leaderboards.^[9]

Phase 4: Frontend Development (Week 5)

React Setup

Create React app with React Router for navigation. Build authentication pages with form validation. Design battle arena with real-time rap display and turn indicators.^[8]

WebSocket Integration

Connect Socket.IO client to battle service. Display AI responses in real-time as they're generated. Show live scoring and leaderboard updates.^[7]

ShadCN UI

Integrate ShadCN components for beautiful UI (since you love it). Create battle cards, leaderboard tables, and user profiles.^[8]

Phase 5: Music Generation (Week 6)

Music Service

Integrate **Mubert API** or **Riffusion** for beat generation. Allow users to select beat styles (Trap, Boom Bap, Lo-fi). Cache generated beats in cloud storage (Cloudinary free tier).^[4]

TTS Integration

Add **Coqui TTS** or **Bark** for converting AI lyrics to speech. Process audio in background and stream to frontend. Store generated audio files temporarily.^[5]

Phase 6: Nginx Load Balancing (Week 7)

Nginx Configuration

Create `nginx.conf` to define upstream servers for each microservice. Configure `proxy_pass` to route requests to appropriate services.^[10] ^[9]

```
upstream auth_service {
    least_conn;
    server auth-service-1:5001;
    server auth-service-2:5001;
}

upstream battle_service {
    least_conn;
    server battle-service-1:5002;
    server battle-service-2:5002;
}

server {
    listen 80;

    location /api/auth {
        proxy_pass http://auth_service;
    }
}
```

```
location /api/battle {
    proxy_pass http://battle_service;
}
}
```

Docker Compose for Multiple Instances

Run 2-3 instances of each service and let Nginx distribute load.^[10] ^[9]

Phase 7: Dockerization (Week 8)

Create Dockerfiles

Write Dockerfile for each service with optimized builds. Create multi-stage builds to reduce image size.^[7]

Docker Compose

Update `docker-compose.yml` to orchestrate all services. Add health checks for each service. Set up volume mounting for MongoDB data persistence.^[7]

Phase 8: Kubernetes Deployment (Week 9-10)

Create Kubernetes Manifests

Write deployment YAML files for each microservice. Configure service YAML files for internal communication. Add ConfigMaps for environment variables and Nginx configuration.^[11] ^[12]

Auto-Scaling

Configure Horizontal Pod Autoscaler for AI service (scales based on CPU). Set resource limits and requests for each pod.^[12] ^[11]

Ingress Controller

Deploy Nginx Ingress Controller to route external traffic. Configure TLS/SSL certificates for production.^[13]

Phase 9: Testing & Optimization (Week 11)

Load Testing

Use Apache JMeter or k6 to simulate 100+ concurrent battles. Monitor Nginx load distribution across service instances. Test Kafka throughput and RabbitMQ message delivery.^[9] ^[7]

Performance Optimization

Add Redis caching for frequently accessed data. Optimize MongoDB queries with indexes. Compress API responses with gzip in Nginx.^[9]

Phase 10: Additional Features (Week 12+)

Tournament System: Create scheduled rap battles with bracket systems. **Replay Feature:** Use Kafka event logs to replay entire battles. **Voice Input:** Add browser speech recognition for users to rap into microphone. **Social Features:** Share battle highlights on social media.

Development Tips

Start with **2 microservices** (auth + battle) to avoid complexity. Add more services gradually as you learn each technology. Use **Docker Compose** for local development before moving to Kubernetes. Deploy to **free Kubernetes** platforms like Google Cloud free tier or Oracle Cloud for testing. Keep all AI models **free** initially—upgrade to paid APIs once app gains users.^{[3] [11] [8] [7]}

This project will take 3-4 months to complete properly, but it'll teach you production-grade skills companies pay 24+ lakhs for!^{[9] [7]}



1. <https://www.semrush.com/blog/chatgpt-alternatives/>
2. <https://www.appvizer.co.uk/magazine/artificial-intelligence/ai-assistant/alternative-chat-gpt>
3. <https://www.technaureus.com/blog-detail/5-free-ai-tools-for-startups-in-2025>
4. <https://www.digitalocean.com/resources/articles/ai-music-generators>
5. <https://lablab.ai/event/execute-ai-genesis/idealginiters/freestyle-ai-battle-arena>
6. <https://stackoverflow.com/questions/51126472/how-to-organise-file-structure-of-backend-and-frontend-in-mern>
7. <https://dev.to/davydocsurg/mastering-microservices-a-hands-on-tutorial-with-nodejs-rabbitmq-nginx-and-docker-m4f>
8. <https://dev.to/kingsley/mern-stack-project-structure-best-practices-2adk>
9. <https://blog.stackademic.com/optimizing-microservices-with-nginx-for-efficient-load-balancing-314d43eb7064>
10. <https://www.theserverside.com/blog/Coffee-Talk-Java-News-Stories-and-Opinions/How-to-setup-an-Nginx-load-balancer-example>
11. <https://www.geeksforgeeks.org/blogs/top-kubernetes-project-ideas-for-beginners/>
12. <https://github.com/techiescamp/kubernetes-projects>
13. <https://www.f5.com/company/blog/nginx/nginx-tutorial-deploy-configure-microservices>
14. https://www.reddit.com/r/github/comments/ptu8cj/how_should_i_set_up_a_mern_stackfolder_structure/
15. <https://www.squareinfosoft.com/best-practices-for-mern-stack-development/>
16. <https://www.zignuts.com/blog/mern-stack-2025-full-stack-apps-guide>
17. <https://github.com/paritosh9199/mern-microservices>
18. <https://zapier.com/blog/chatgpt-alternatives/>
19. https://nginx.org/en/docs/http/load_balancing.html
20. <https://www.accuwebhosting.com/blog/chatgpt-alternatives/>
21. <https://30dayscoding.com/blog/load-balancing-for-microservices-nginx-haproxy>
22. <https://wotnot.io/blog/chatgpt-alternatives>
23. <https://docs.nginx.com/nginx/admin-guide/load-balancer/http-load-balancer/>
24. <https://www.synthesia.io/post/ai-tools>